Secure Code Review Assistant

AI-Powered Debugging and Vulnerability Detection

Technical Report

Submitted by: Mrinmayee Pramod Kulkarni | **PRN:** 124B2B025
Department: Computer Engineering | **College:** PCCOE, Pune (2026–2027)

1. Introduction

The Secure Code Review Assistant is a web-based application developed using Python and Streamlit. It helps users review source code and identify potential security vulnerabilities, programming errors, and areas for improvement. The application uses the Google Gemini API to analyze the submitted code and generate suggestions.

2. Problem Statement

Manual code review can take time, especially when checking code for common security issues and programming mistakes. Developers and students may overlook vulnerabilities when reviewing code. This project provides a simple interface to support code review and help users understand potential issues.

3. Objectives

- Analyze source code for potential security vulnerabilities.
- Identify possible programming errors and unsafe coding practices.
- Provide explanations and suggestions for improvement.
- Offer a simple interface for submitting code and viewing results.
- Allow users to download the analysis report.

4. Technologies Used

- **Python:** Main programming language.
- **Streamlit:** Web interface.
- **Google Gemini API:** Code analysis and recommendations.
- **google-genai:** Python SDK for accessing Gemini.
- **python-dotenv:** Loads configuration values from the .env file.

5. System Architecture

The application follows a simple workflow:

1. The user opens the Streamlit application.
2. The user selects a programming language and enters source code.
3. The application sends the code to the analysis module.
4. The analysis module submits the code to the Gemini API.
5. The response is displayed in the application.
6. The user reviews the findings and can download the report.

6. Main Modules

6.1 User Interface (app.py)

This module provides the application interface. It accepts source code, allows language selection, starts the analysis, displays the results, and provides the report download option.

6.2 Code Analysis (analyzer.py)

This module connects the application to the Gemini API. It sends the submitted code for analysis and returns the generated findings and recommendations.

6.3 Configuration (.env)

The environment file stores the Gemini API key and model configuration. It is kept separate from the source code to avoid exposing credentials.

7. Implementation

The application was developed in Python. Streamlit was used to create the interface, while the Google Gen AI SDK was used to communicate with Gemini. The user submits code through the interface, and the analysis results are displayed for review.

The project can be started from its directory using the command:

```
streamlit run app.py
```

The Gemini API key must be configured correctly in the local environment before analysis can be performed.

8. Testing and Results

The application was tested using a Python function that constructs an SQL query through string concatenation:

```
query = "SELECT * FROM users WHERE id = " + user_id
```

The analysis identified the potential risk of SQL injection and recommended using parameterized SQL queries. This example demonstrated how the application can help users recognize unsafe coding practices.

The results depend on the submitted code, API availability, and the model's response. The findings should be reviewed before making changes to an application.

9. Limitations

- The analysis depends on the availability of the Gemini API.
- Results may contain false positives or miss certain issues.
- The application does not replace professional security testing.
- The quality of the results depends on the submitted code and context.
- API usage may be subject to account quotas and applicable charges.

10. Future Scope

The application could be extended with support for more programming languages, automated code quality checks, severity-based vulnerability classification, and integration with version-control platforms. Additional testing could also improve the consistency of the analysis.

11. Conclusion

The Secure Code Review Assistant provides a simple way to examine source code for potential security vulnerabilities and programming issues. By combining Python, Streamlit, and Google Gemini, the project demonstrates how an AI-powered service can assist students and developers during code review. The generated findings should be treated as recommendations and verified before implementation.

Name and Signature: Mrinmayee Pramod Kulkarni

Date: 15/10/2026